

```
#include<iostream>
#include<cstdlib>
#include<ctime>
using namespace std;
```

```
#define SIZE 10
```

```
#define UP 0
#define DOWN 1
#define LEFT 2
#define RIGHT 3
```

```
class Node
{
public:
    Node()
    {
        Node(0, 0);
    }
    Node(int r, int c)
    {
        row = r;
        col = c;
    }
    int getRow() { return row; }
    int getCol() { return col; }
    void setRow(int r)
    {
        if(r >= 0 && r < SIZE)
            row = r;
    }
    void setCol(int c)
    {
        if(c >= 0 && c < SIZE)
            col = c;
    }
private:
    int col, row;
};
```

```
class List
{
public:
    List()
    {
        top = 0;
    }

    /*
    Insert an element from top
    */
    void addElement(int r, int c)
    {
        if (top < SIZE * SIZE) // Ensure not to exceed the list's size
        {
            data[top][0] = r; // Row
            data[top][1] = c; // Column
        }
    }
};
```

```

        top++;
    }
}

/*
remove an element from top and return a pointer point to the element.
If list is empty, return NULL.
*/
void removeElement()
{
    if (top > 0)
    {
        top--;
    }
}

// Print the path by showing the directions between the grid cells
void printPath()
{
    for (int i = 1; i < top; i++)
    {
        if (data[i][0] == data[i - 1][0] - 1 && data[i][1] == data[i - 1][1])
            cout << "UP\n";
        else if (data[i][0] == data[i - 1][0] + 1 && data[i][1] == data[i - 1][1])
            cout << "DOWN\n";
        else if (data[i][1] == data[i - 1][1] - 1 && data[i][0] == data[i - 1][0])
            cout << "LEFT\n";
        else if (data[i][1] == data[i - 1][1] + 1 && data[i][0] == data[i - 1][0])
            cout << "RIGHT\n";
    }
}

private:
    int data[SIZE * SIZE][2]; // Array to store the row and column of the path
    int top;
};

class Maze
{
public:
    Maze()
    {
        initMaze(SIZE);
    }

    /*
function initMaze
Allocate a 2-D array with s * s sizes as the map of maze.
Inside the maze where 0 represents empty space and 1 represents a wall.
[0][0] is start point and [s - 1][s - 1] is finish point.
Randomly generate 20% wall in the maze.
Make sure [0][0] and [s - 1][s - 1] are 0
*/
    void initMaze(int s)
    {
        maze = new int*[s];
        for (int i = 0; i < s; i++)
        {
            maze[i] = new int[s];
        }
    }
}

```

```

// Initialize random walls, start, and end points
srand(time(0));
for (int i = 0; i < s; i++)
{
    for (int j = 0; j < s; j++)
    {
        maze[i][j] = 0;
    }
}
// Randomly generate walls (1)
int wallCount = (s * s) * 0.2; // Calculate 20% of total spaces for walls
while (wallCount > 0) {
    int r = rand() % s;
    int c = rand() % s;

    // Ensure that we do not place walls at start and finish points
    if ((r != 0 || c != 0) && (r != s - 1 || c != s - 1) && maze[r][c] == 0) {
        maze[r][c] = 1; // Place a wall
        wallCount--;
    }
}
}

```

```

/*
function getPath
This function will find a path between start point and finish point.
Return a list content the path information inside.
If there is no path between two point, then the list will be empty.
*/
List* getPath()
{
    List* path = new List();
    bool visited[SIZE][SIZE] = {false}; // Array to track visited cells
    findPath(0, 0, visited, path);
    return path;
}

```

```

void printMaze()
{
    for (int i = 0; i < SIZE; i++)
    {
        for (int j = 0; j < SIZE; j++)
        {
            if (maze[i][j] == 0)
                cout << " ";
            else if (maze[i][j] == 1)
                cout << "*";
        }
        cout << endl;
    }
}

```

```

private:
int** maze;

```

```

// Recursive function to find a path from start to finish
bool findPath(int r, int c, bool visited[SIZE][SIZE], List* path)
{

```

```

// Base cases
if (r < 0 || c < 0 || r >= SIZE || c >= SIZE || maze[r][c] == 1 || visited[r][c])
    return false; // Out of bounds, wall, or already visited

// Mark the cell as visited
visited[r][c] = true;

// Add the cell to the path
path->addElement(r, c);

// If reached the end point, return true
if (r == SIZE - 1 && c == SIZE - 1)
    return true;

// Explore neighbors (right, down, left, up)
if (findPath(r, c + 1, visited, path) ||
    findPath(r + 1, c, visited, path) ||
    findPath(r, c - 1, visited, path) ||
    findPath(r - 1, c, visited, path))
{
    return true;
}

// Backtrack if no path is found
path->removeElement();
return false;
}
};

int main()
{
    Maze* maze = new Maze();
    maze->printMaze();
    maze->getPath()->printPath();
}

```